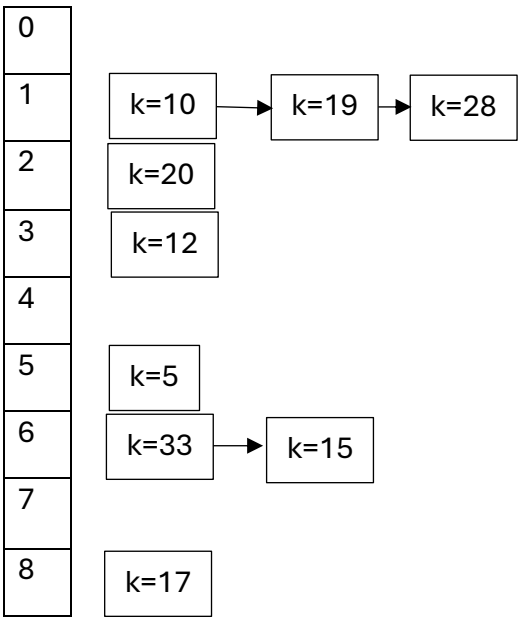
# Hash Tables
# Parte 1

## Ejercicio 1:

**Ejemplificar que pasa cuando insertamos las llaves 5, 28, 19, 15, 20, 33, 12, 17, 10 en un HashTable con la colisión resulta por el método de chaining. Permita que la tabla tenga 9 slots y la función de hash:**

$$H(k) = k \; mod \; 9$$




# Parte 2

## Ejercicio 3:

**Considerar una tabla hash de tamaño m = 1000 y una función de hash correspondiente al método de la multiplicación donde A = (sqrt(5)-1)/2). Calcular las ubicaciones para las claves 61, 62, 63, 64 y 65.**

La función hash quedaría definida así:

$$h(k) = \left\lfloor 1000 \left( \left( k \frac{\sqrt{5} - 1}{2} \right) mod \; 1 \right) \right\rfloor$$

Evaluando las claves dadas, las posiciones serían las siguientes:

- h(61) = 700
- h(62) = 318
- h(63) = 936
- h(64) = 554
- h(65) = 172

## Ejercicio 4:

**Implemente un algoritmo lo más eficiente posible que devuelva True o False a la siguiente proposición: dado dos strings $s_1...s_k$ y $p_1...p_k$, se quiere encontrar si los caracteres de $p_1...p_k$ corresponden a una permutación de $s_1...s_k$. Justificar el costo en tiempo de la solución propuesta.**

En el caso promedio, el costo esperado de cada búsqueda e inserción es de O(1), con n iteraciones en cada bucle, la complejidad queda de O(n).

En el peor caso, si todas las claves colisionan, la lista enlazada interna crece linealmente. Por lo tanto, en el paso i, buscar e insertar toma O(i), así la complejidad total es de $O(n^2)$

## Ejercicio 5:

**Implemente un algoritmo que devuelva True si la lista que recibe de entrada tiene todos sus elementos únicos, y Falso en caso contrario. Justificar el costo en tiempo de la solución propuesta.**

Caso promedio: O(n), ya que cada búsqueda e inserción tiene de costo O(1) esperado por el uso de hashing.

Peor caso: $O(n^2)$ si todas las claves colisionan y además todos los elementos de la lista son únicos.

## Ejercicio 6:

**Los nuevos códigos postales argentinos tienen la forma cddddccc, donde c indica un carácter (A - Z) y d indica un dígito 0, . . . , 9. Por ejemplo, C1024CWN es el código postal que representa a la calle XXXX a la altura 1024 en la Ciudad de Mendoza. Encontrar e implementar una función de hash apropiada para los códigos postales argentinos.**

Implementado en código

## Ejercicio 7:

**Implemente un algoritmo para realizar la compresión básica de cadenas utilizando el recuento de caracteres repetidos. Por ejemplo, la cadena *'aabccccccaaa'* se convertiría en 'a2blc5a3'. Si la cadena "comprimida" no se vuelve más pequeña que la cadena original, su método debería devolver la cadena original. Puedes asumir que la cadena sólo tiene letras mayúsculas y minúsculas (a - z, A - Z). Justificar el costo en tiempo de la solución propuesta.**

El algoritmo propuesto tiene de costo computacional O(n), siendo n la longitud de la cadena, ya que debe recorrerla completa dos veces, una para armar el hash table y otra para comprimirla.

## Ejercicio 8:

**Se requiere encontrar la primera ocurrencia de un string $p_1...p_k$ en uno más largo $a_1...a_L$. Implementar esta estrategia de la forma más eficiente posible con un costo computacional menor a O(K*L) (solución por fuerza bruta). Justificar el coste en tiempo de la solución propuesta.**

El algoritmo busca la primera aparición de p dentro de a. Calcula un hash para p y para una ventana inicial de a del mismo largo; luego desplaza esa ventana, actualizando su hash en tiempo constante. Cuando los hashes coinciden, compara los caracteres para confirmar que las cadenas sean iguales. Devuelve la posición encontrada o -1 si no hay coincidencia.

El cálculo inicial cuesta O(m) y el recorrido de las ventanas, O(n), donde n = es la longitud de a y m es la longitud de p. Por eso, el costo promedio es de O(n + m). En el peor caso puede ser O(nm), porque varias ventanas podrían tener el mismo hash y requerir una comparación carácter por carácter.

## Ejercicio 9:

**Considerar los conjuntos de enteros S = {s1, . . . , sn} y T = {t1, . . . , tm}. Implemente un algoritmo que utilice una tabla de hash para determinar si S ⊆ T (S subconjunto de T). ¿Cuál es la complejidad temporal del caso promedio del algoritmo propuesto?**

La complejidad temporal de este algoritmo es O(n + m), donde n es el tamaño de S y m es el tamaño de T. Esto se debe a que se recorre la lista T para insertar sus elementos en la tabla hash, y luego se recorre la lista S para verificar si cada elemento está presente en la tabla hash.

# Parte 3

## Ejercicio 10:

**Considerar la inserción de las siguientes llaves: 10; 22; 31; 4; 15; 28; 17; 88; 59 en una tabla hash de longitud m = 11 utilizando direccionamiento abierto con una función de hash *h'(k) = k*. Mostrar el resultado de insertar estas llaves utilizando:**

1. **Linear probing:** (k + i) mod 11

| **Índice** | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9** | **10** |
|---|---|---|---|---|---|---|---|---|---|---|---|
| k | 22 | 88 | | | 4 | 15 | 28 | 17 | 59 | 31 | 10 |

2. **Quadratic probing con *c1 = 1* y c2 = 3** $(k + i + 3i^2) \bmod 11$

| **Índice** | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9** | **10** |
|---|---|---|---|---|---|---|---|---|---|---|---|
| k | 22 | | 88 | 17 | 4 | | 28 | 59 | 15 | 31 | 10 |

3. **Double hashing con *h1(k) = k* y *h2(k) = 1 +(k mod (m - 1)):*** $(k + i (1 + (k \bmod 10))) \bmod 11$

| **Índice** | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9** | **10** |
|---|---|---|---|---|---|---|---|---|---|---|---|
| k | 22 | | 59 | 17 | 4 | 15 | 28 | 88 | | 31 | 10 |

## Ejercicio 12:

**Las llaves 12, 18, 13, 2, 3, 23, 5 y 15 se insertan en una tabla hash inicialmente vacía de longitud 10 utilizando direccionamiento abierto con función hash h(k) = k mod 10 y exploración lineal (linear probing). ¿Cuál es la tabla hash resultante? Justifique.**

| | |
|---|---|
| 0 | |
| 1 | |
| 2 | 2 |
| 3 | 23 |
| 4 | |
| 5 | 15 |
| 6 | |
| 7 | |
| 8 | 18 |
| 9 | |

(A)

| | |
|---|---|
| 0 | |
| 1 | |
| 2 | 12 |
| 3 | 13 |
| 4 | |
| 5 | 5 |
| 6 | |
| 7 | |
| 8 | 18 |
| 9 | |

(B)

| | |
|---|---|
| 0 | |
| 1 | |
| 2 | 12 |
| 3 | 13 |
| 4 | 2 |
| 5 | 3 |
| 6 | 23 |
| 7 | 5 |
| 8 | 18 |
| 9 | 15 |

(C)

| | |
|---|---|
| 0 | |
| 1 | |
| 2 | 12, 2 |
| 3 | 13, 3, 23 |
| 4 | |
| 5 | 5, 15 |
| 6 | |
| 7 | |
| 8 | 18 |
| 9 | |

(D)

La tabla hash resultante sería la C, ya que en la A y en la B no se encuentran todos los elementos que se deseaban insertar, y en la D se solucionan las colisiones con encadenamiento y no con linear probing.

## Ejercicio 13:

**Una tabla hash de longitud 10 utiliza direccionamiento abierto con función hash h(k)=k mod 10, y exploración lineal (linear probing). Después de insertar 6 valores en una tabla hash vacía, la tabla es como se muestra a continuación.**

| | |
|---|---|
| 0 | |
| 1 | |
| 2 | 42 |
| 3 | 23 |
| 4 | 34 |
| 5 | 52 |
| 6 | 46 |
| 7 | 33 |
| 8 | |
| 9 | |

**¿Cuál de las siguientes opciones da un posible orden en el que las llaves podrían haber sido insertadas en la tabla? Justifique**

**(A) 46, 42, 34, 52, 23, 33**

**(B) 34, 42, 23, 52, 33, 46**

**(C) 46, 34, 42, 23, 52, 33**

**(D) 42, 46, 33, 23, 34, 52**

Realizando los distintos órdenes, obtenemos las siguientes tablas hash

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | 42 | 52 | 34 | 23 | 46 | 33 | | |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | 42 | 23 | 34 | 52 | 33 | 46 | | |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | 42 | 23 | 34 | 52 | 46 | 33 | | |

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | 42 | 33 | 23 | 34 | 46 | 52 | | |

Así, la respuesta es la C